Ensemble Learning

Instructor: Dr. Chao Huang

Department of Computer Science
Montclair State University

Outline

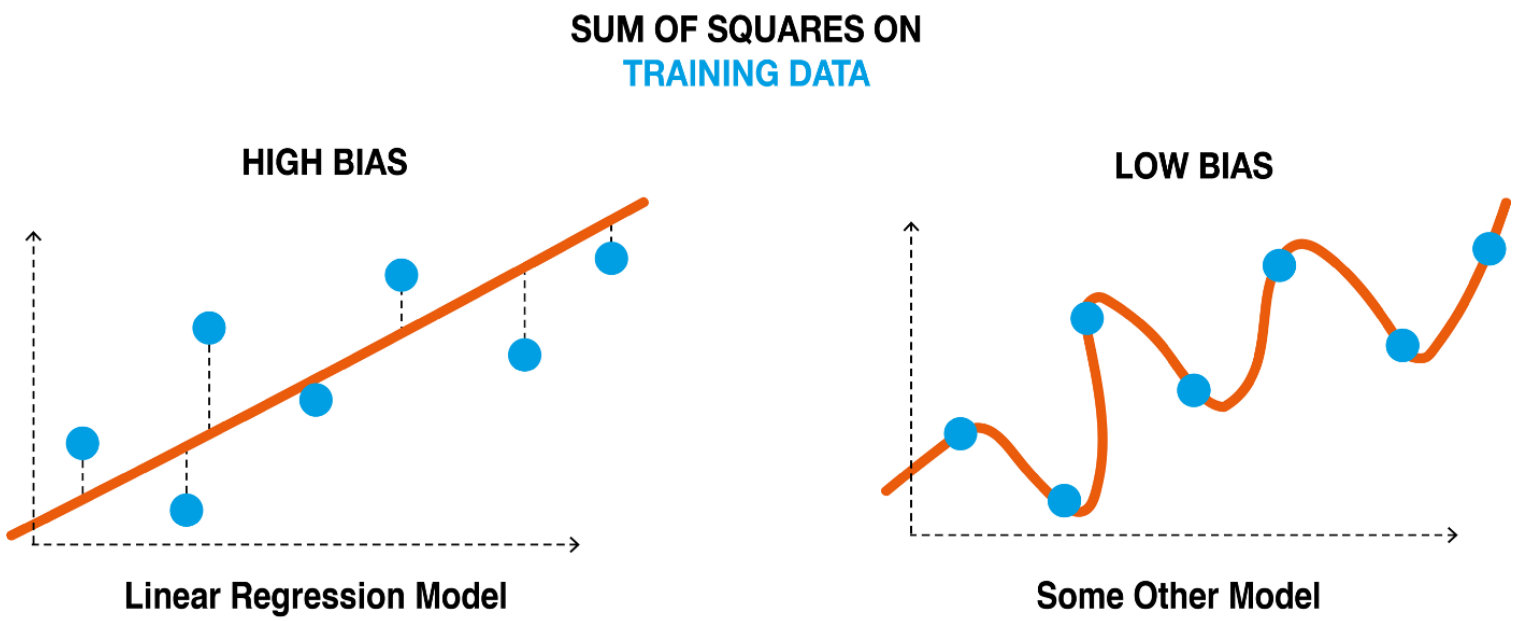
1. Bias-Variance Tradeoff of ML models
2. What is Ensemble Learning and Why?
3. Ensemble Learning Methods: bagging and boosting
4. Summary

Bias

- Total error of an ML model can be decomposed:

$$Error = Bias^2 + Variance + Noise$$

  - Bias: inability of a model to correctly estimate the target

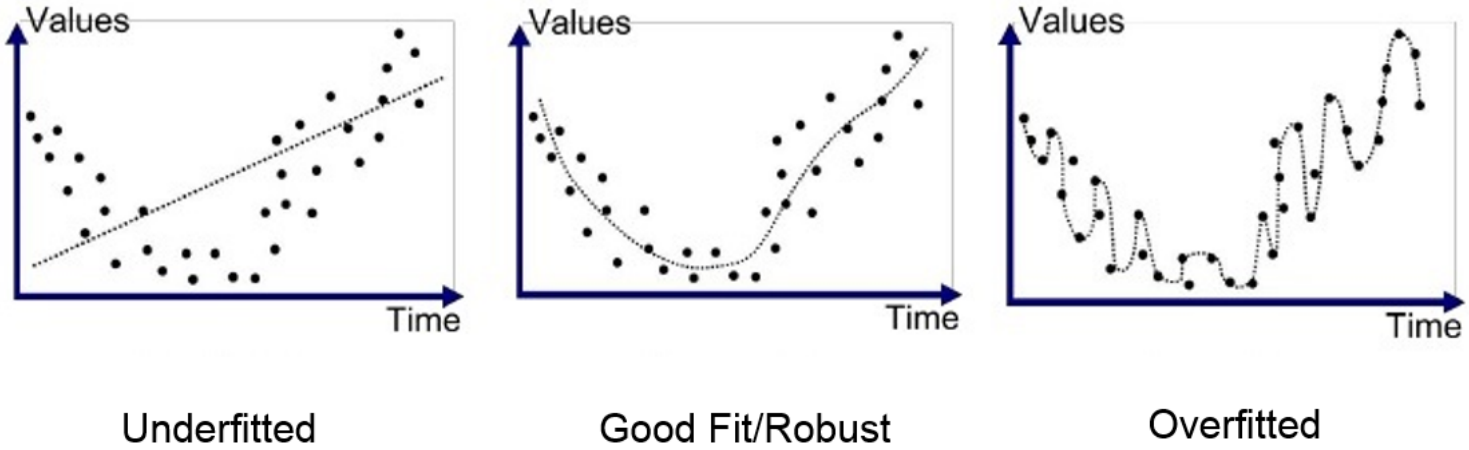


Variance

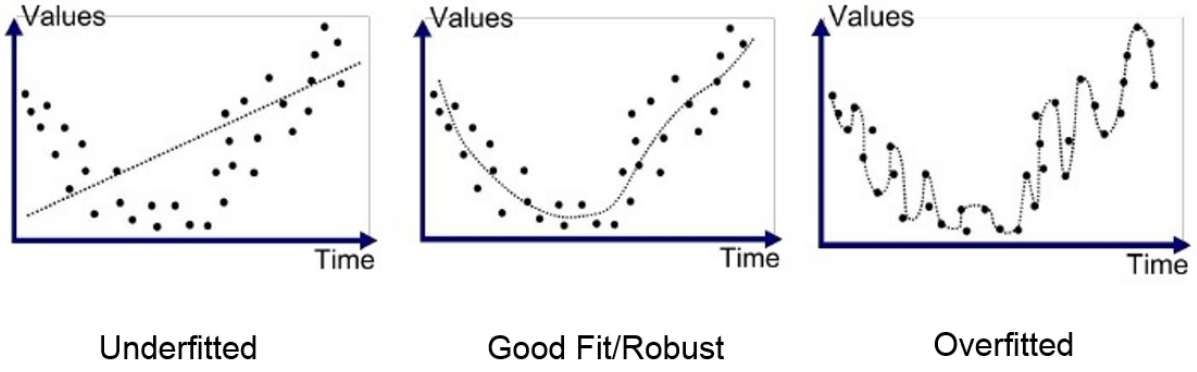
- Total error of an ML model can be decomposed:

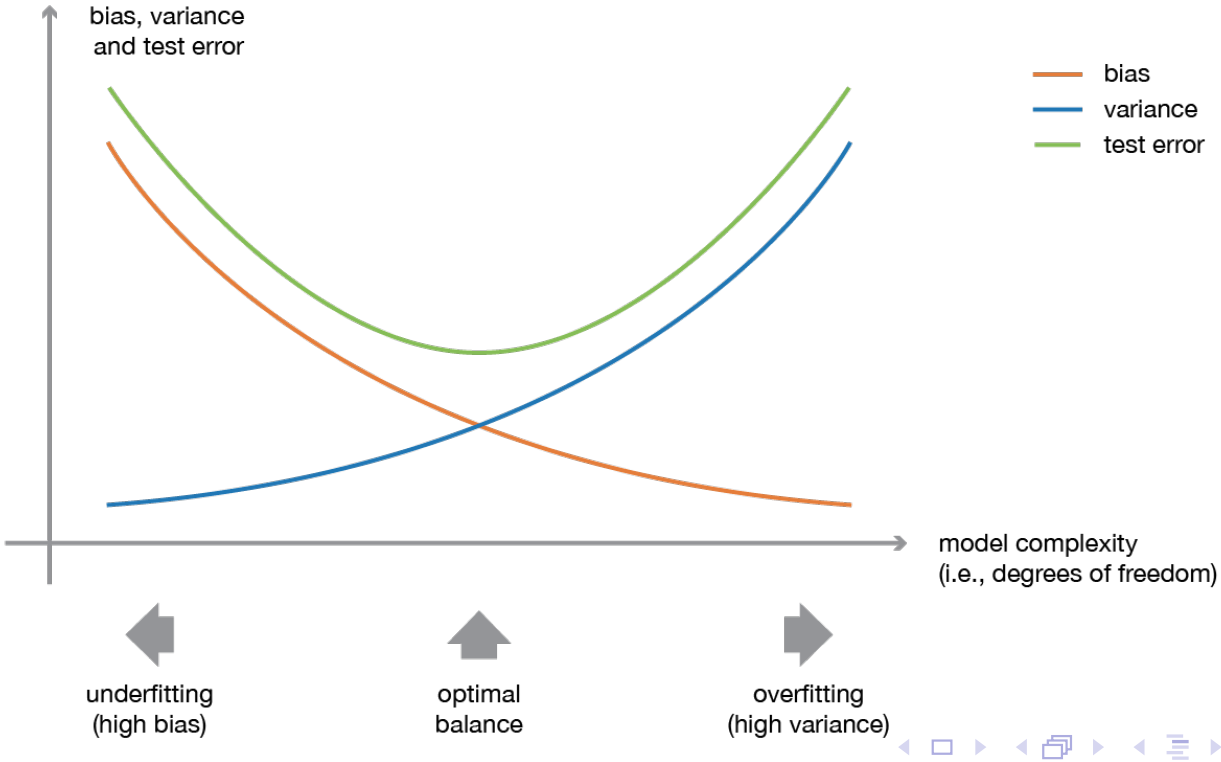
$$Error = Bias^2 + Variance + Noise$$

  - Variance: error that occurs due to a model's sensitivity to small fluctuations in the training set
  - E.g., High variance would model the noise $\rightarrow$ overfitting



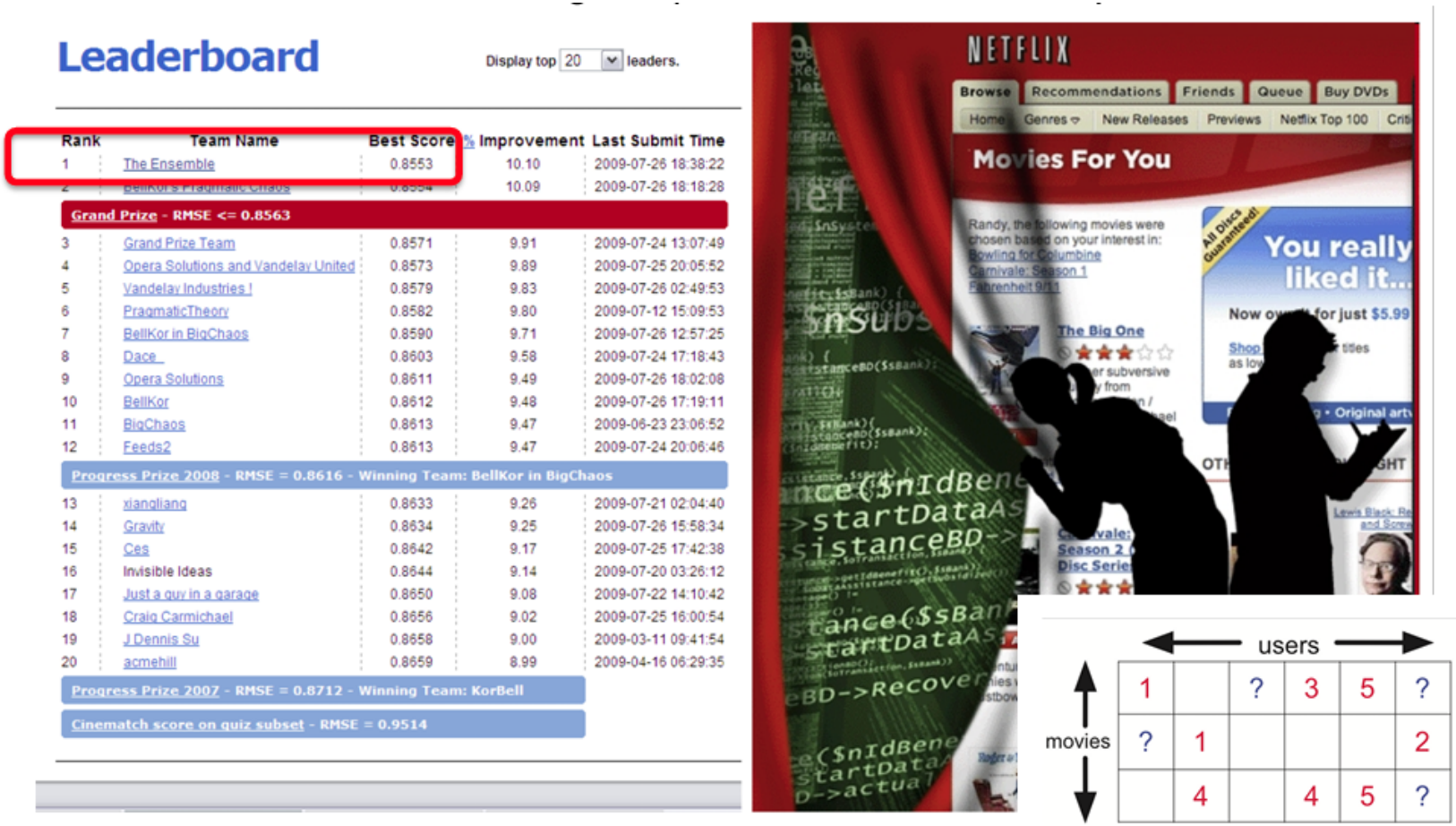
Bias-Variance Tradeoff





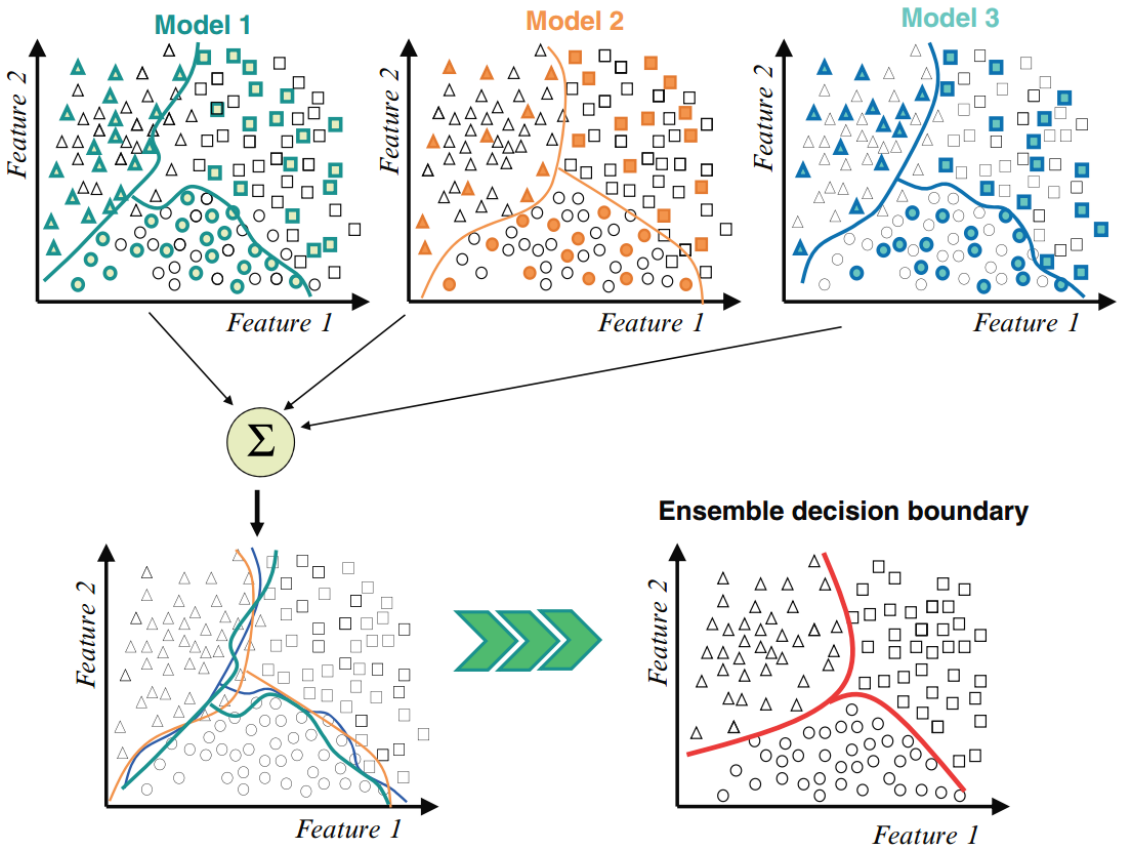
Ensemble Learning

- Wisdom of crowd
  - Proper aggregation of crowd decisions can outperform experts
- **Ensemble learning**: machine learning interpretation
  - Errors of single model compensated by other models → better ensemble



Ensemble Learning: An Illustration

- 3-class classification



Principles of Ensemble Learning

- Predictive performance: the individual inducer's predictive performance should be as high as possible
- Diversity: the participating inducers should be sufficiently diverse in order to gain a desired predictive performance
- Question:
  - One training data set
  - How to create diverse models?

Principles of Ensemble Learning

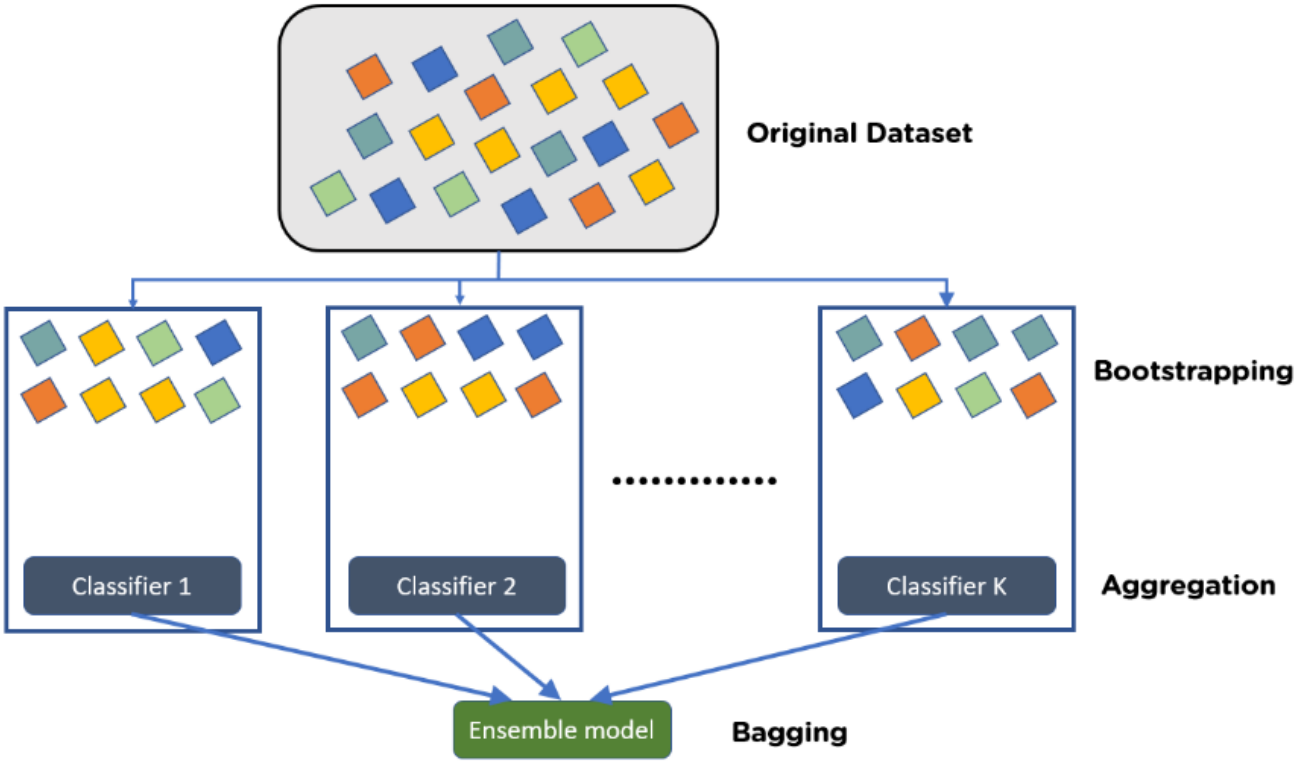
- Predictive performance: the individual inducer's predictive performance should be as high as possible
- Diversity: the participating inducers should be sufficiently diverse in order to gain a desired predictive performance
- Question:
  - One training data set
  - How to create diverse models?

Ensemble Learning Methods

1. **Parallel**: base learners generated in parallel
   - Voting/Averaging
   - Bagging
   - Random forest
   - Stacking
   - .....
2. **Sequential**: output of a base learner affects construction of next one
   - AdaBoost
   - Gradient boost
   - .....

Bagging (Bootstrap aggregating)

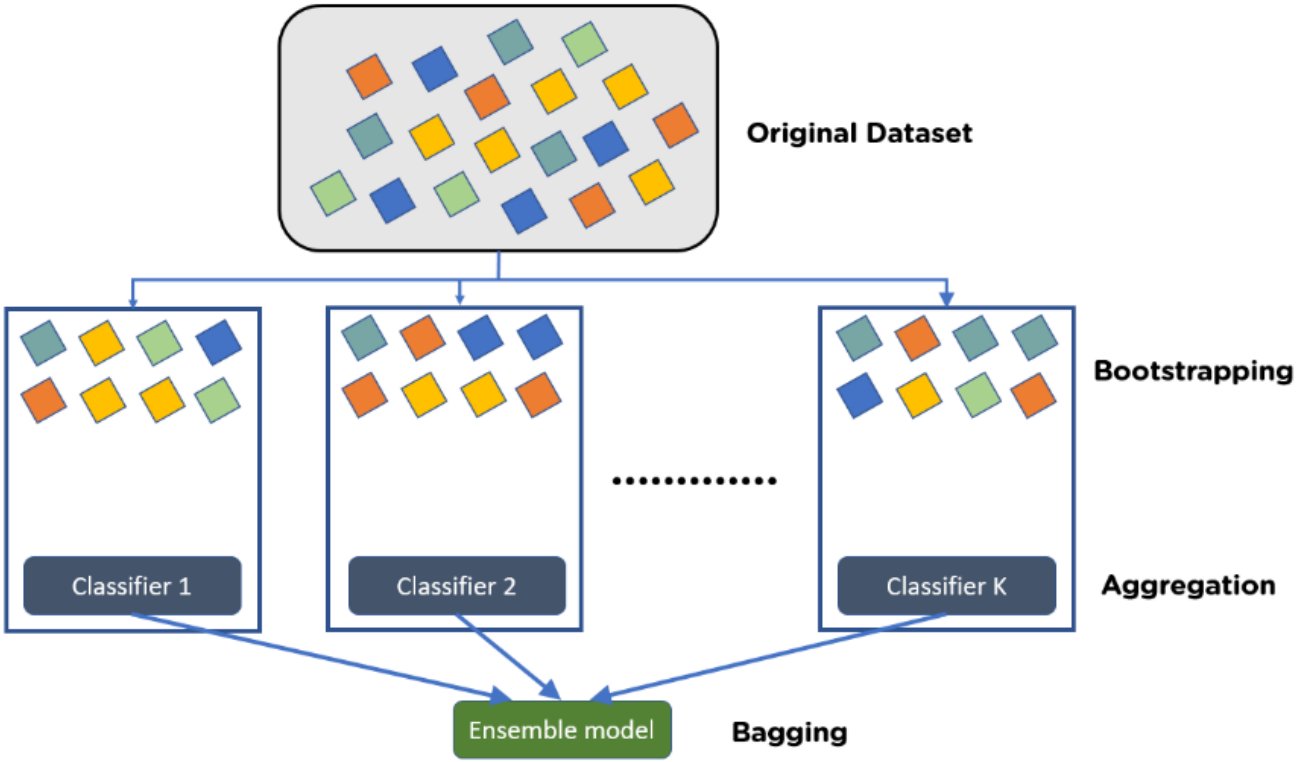
- **Bootstrap**: each learner is trained using data sampled from the original data set with replacement
- **Aggregating**: majority voting
- A parallel method and mainly reduces variance



- Question: where does diversity come from?

Bagging (Bootstrap aggregating)

- **Bootstrap**: each learner is trained using data sampled from the original data set with replacement
- **Aggregating**: majority voting
- A parallel method and mainly reduces variance



- Question: where does diversity come from?

AdaBoost (Adaptive Boosting)

- **Adaptive**: learn from the mistakes from earlier models
  - Each new model focuses on the most difficult observations by weighting training samples
- **Boosting**: enough learning boosts the performance
- A sequential method that mainly reduces bias
- Three-step process
  1. Initialization
  2. Sequential training
  3. Final aggregation

AdaBoost (Adaptive Boosting)

- **Adaptive**: learn from the mistakes from earlier models
  - Each new model focuses on the most difficult observations by weighting training samples
- **Boosting**: enough learning boosts the performance
- A sequential method that mainly reduces bias
- Three-step process
  1. Initialization
  2. Sequential training
  3. Final aggregation

AdaBoost Process

- Dataset with D samples $\{(x_d, y_d)\}_{d=1}^{D}$; N learners
- **1. Initialization**: for learner $i = 1$
  - Each sample d is assigned an equal weight

$$w_d^{(i)} = \frac{1}{D}. \tag{1}$$

AdaBoost Process

- Dataset with D samples $\{(x_d, y_d)\}_{d=1}^{D}$; N learners
- **1. Initialization**: for learner $i = 1$
  - Each sample d is assigned an equal weight

$$w_d^{(i)} = \frac{1}{D}. \tag{1}$$

AdaBoost Process - Cont'd

- **2. Sequential training**: for $i = 1$ to N
  - Train a model on the dataset using current weights $w_d^{(i)}$
  - Compute the weighted error rate of learner i:

$$\epsilon_i = \frac{\sum_{d=1}^{D} w_d^{(i)} \times 1_{y_d \neq h_i(x_d)}}{\sum_{d=1}^{D} w_d^{(i)}} \tag{2}$$

    - ★ $h_i(x_d)$: learner i's prediction on sample d; 1: indicator function
  - Compute learner i's weight in final ensemble

$$v_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right) \tag{3}$$

    - ★ v_i decreases in ϵ_i: more erroneous model has less vote
  - Update the weights of samples for learner $i + 1$:

$$w_d^{(i+1)} = w_d^{(i)} \times \exp(\alpha_i \times 1_{y_d \neq h_i(x_d)}) \tag{4}$$

    - ★ Wrong predicted samples have larger weights

AdaBoost Process - Cont'd

- **2. Sequential training**: for $i = 1$ to N
  - Train a model on the dataset using current weights $w_d^{(i)}$
  - Compute the weighted error rate of learner i:

$$\epsilon_i = \frac{\sum_{d=1}^{D} w_d^{(i)} \times 1_{y_d \neq h_i(x_d)}}{\sum_{d=1}^{D} w_d^{(i)}} \tag{2}$$

    - ★ $h_i(x_d)$: learner i's prediction on sample d; 1: indicator function
  - Compute learner i's weight in final ensemble

$$v_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right) \tag{3}$$

    - ★ v_i decreases in ϵ_i: more erroneous model has less vote
  - Update the weights of samples for learner $i + 1$:

$$w_d^{(i+1)} = w_d^{(i)} \times \exp(\alpha_i \times 1_{y_d \neq h_i(x_d)}) \tag{4}$$

    - ★ Wrong predicted samples have larger weights

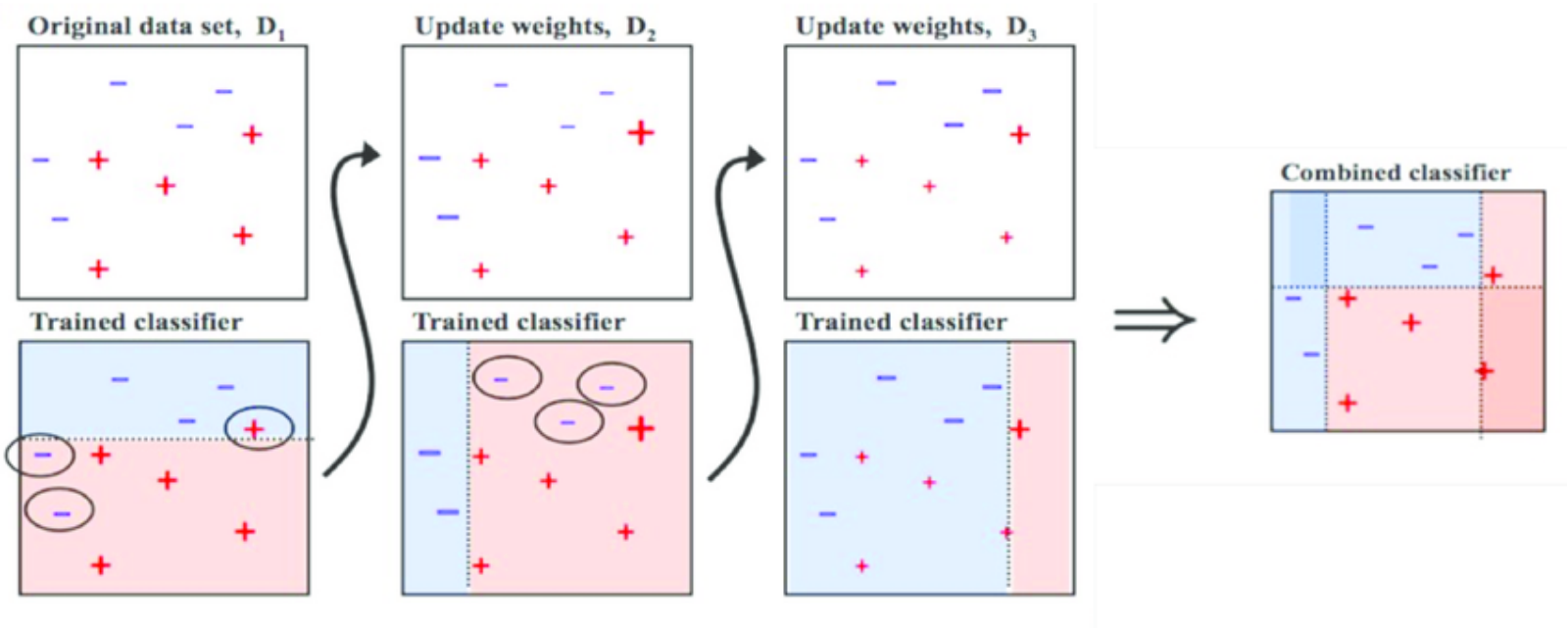
AdaBoost Process - Cont'd

- **3. Final aggregation**: the final ensemble model's prediction is a weighted combination of the learners

$$H(x) = \text{sign}\left(\sum_{i=1}^{N} v_i \times h_i(x)\right) \tag{5}$$

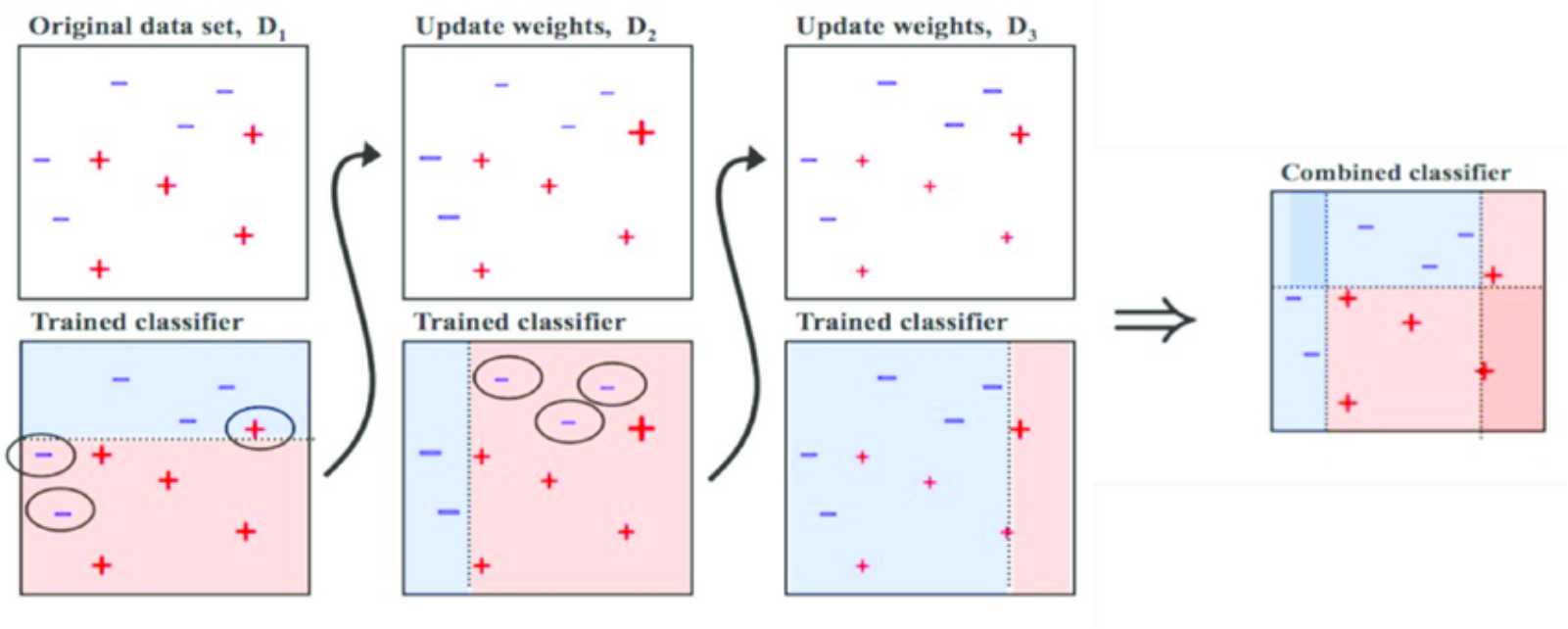
  - "sign" function returns $+1$ if argument is non-negative; -1 otherwise

AdaBoost: An Illustration



- Question: where does diversity come from?

AdaBoost: An Illustration



- Question: where does diversity come from?

Pros and Cons of Ensemble Methods

Pros

- Reduces bias and variance errors
- Local minimum avoidance
- Mitigates class imbalance
  - E.g., each base learner is trained using a balanced subset
- Curse of dimensionality (feature set partitioning)
- .....

Cons

- Computational burden
- Less interpretable than single model
- .....

Pros and Cons of Ensemble Methods

Pros

- Reduces bias and variance errors
- Local minimum avoidance
- Mitigates class imbalance
  - E.g., each base learner is trained using a balanced subset
- Curse of dimensionality (feature set partitioning)
- .....

Cons

- Computational burden
- Less interpretable than single model
- .....

Pros and Cons of Ensemble Methods

Pros

- Reduces bias and variance errors
- Local minimum avoidance
- Mitigates class imbalance
  - E.g., each base learner is trained using a balanced subset
- Curse of dimensionality (feature set partitioning)
- .....

Cons

- Computational burden
- Less interpretable than single model
- .....

Summary

1. Bias-variance tradeoff of ML models
2. Ensemble learning
   - "Wisdom of crowd"
   - Diversity is key
   - Bagging (reduce variance) and AdaBoost (reduce bias)
   - Pros and cons

After-Class Reading

- M¨uller A C, Guido S. Introduction to machine learning with Python: a guide for data scientists. " O'Reilly Media, Inc.", 2016.
  - Sec. 2.3.6
- Zhou, Zhi-Hua. "Ensemble learning." Machine learning. Springer, Singapore, 2021. 181-210.
  - Sec. 8.1-8.5

Appendix: Bagging - Implementation

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
```

In [4]:
```
# Load data (using Iris dataset as an example)
iris = load_iris()
X, y = iris.data, iris.target

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a base estimator
base_estimator = DecisionTreeClassifier()

# Initialize Bagging Classifier
bagging_clf = BaggingClassifier(base_estimator=base_estimator, n_estimators=10, random_state=42)

# Train the model
bagging_clf.fit(X_train, y_train)

# Predict and evaluate the model
predictions = bagging_clf.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy}")
```

```
Accuracy: 1.0
```

Appendix: AdaBoost - Implementation

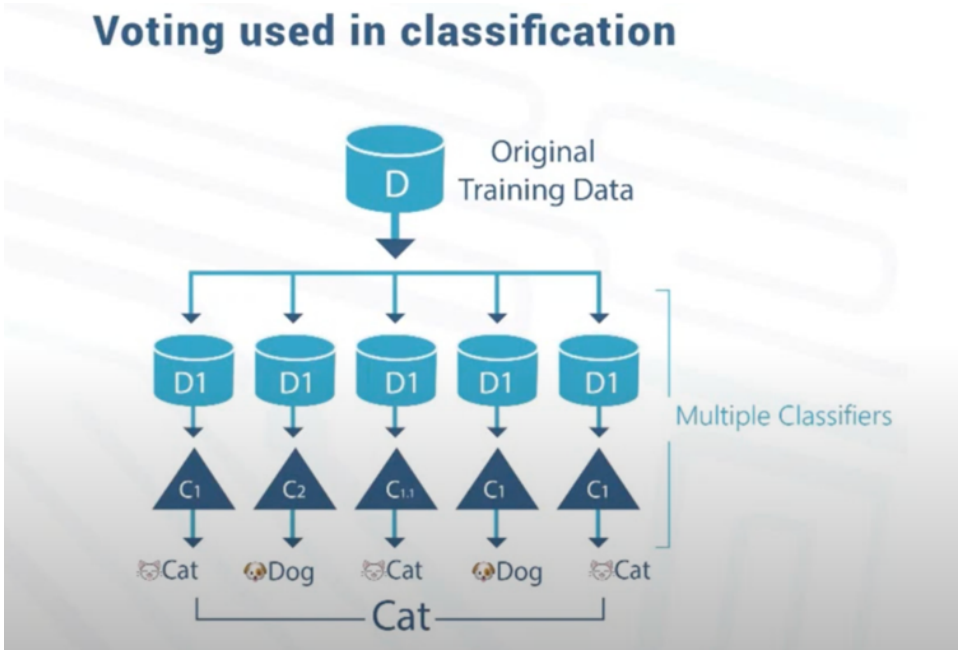
In [5]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
```

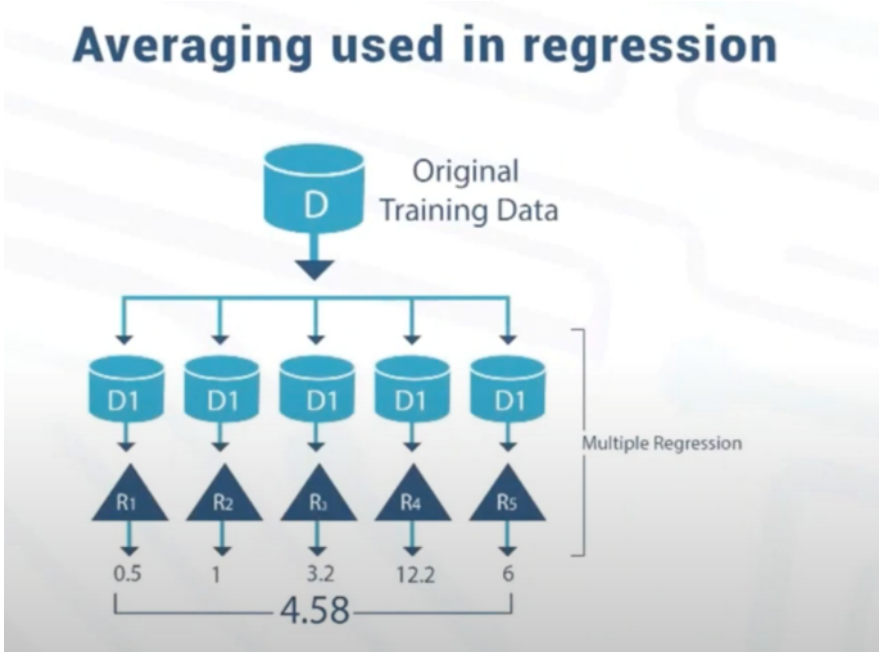
In [6]:
```
# Load data (using Iris dataset as an example)
iris = load_iris()
X, y = iris.data, iris.target

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a base estimator (optional, AdaBoost uses DecisionTreeClassifier as default)
base_estimator = DecisionTreeClassifier(max_depth=1)

# Initialize AdaBoost Classifier
ada_clf = AdaBoostClassifier(base_estimator=base_estimator, n_estimators=100, random_state=42)

# Train the model
ada_clf.fit(X_train, y_train)

# Predict and evaluate the model
predictions = ada_clf.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy}")
```

```
Accuracy: 1.0
```

Appendix: Voting/Averaging

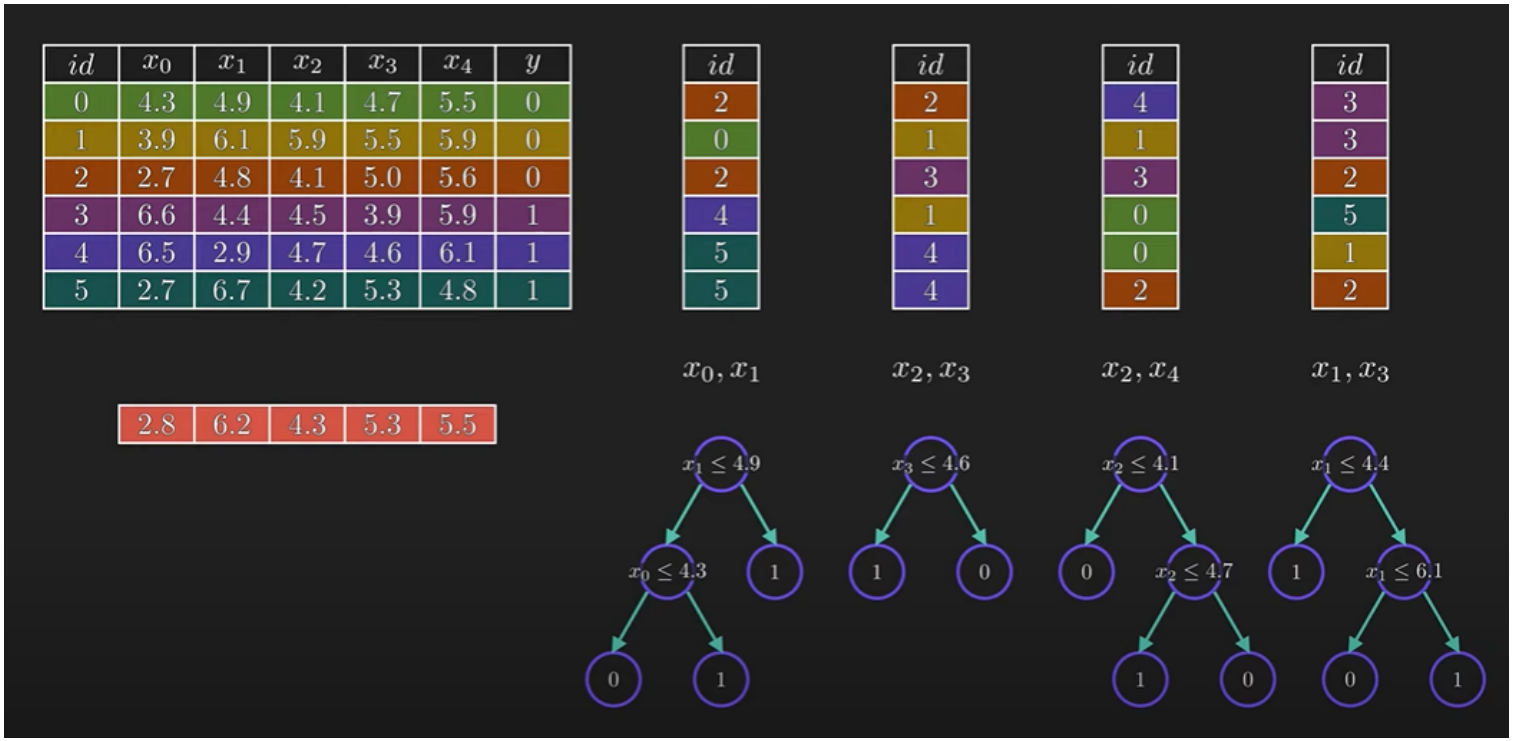
- Same data for each base learner





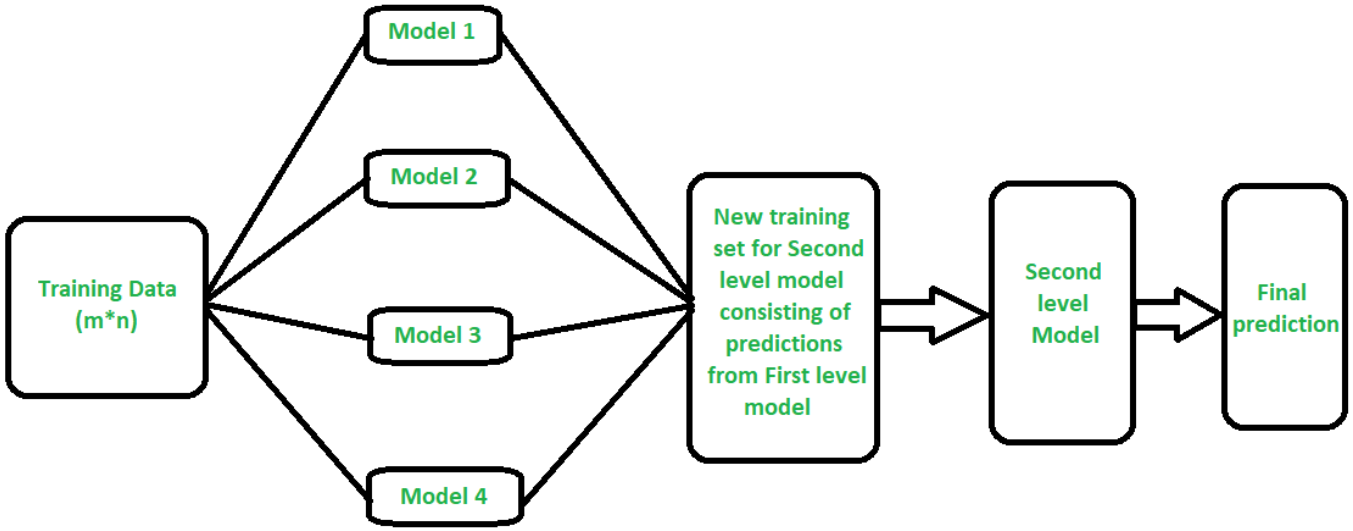
Appendix: Random Forest

- Base learners are decision trees $\rightarrow$ ensemble becomes **forest**
- **Randomness**
  - Bootstrapped samples
  - Random subset of features $\rightarrow$ less correlation
- A parallel method and mainly reduces variance



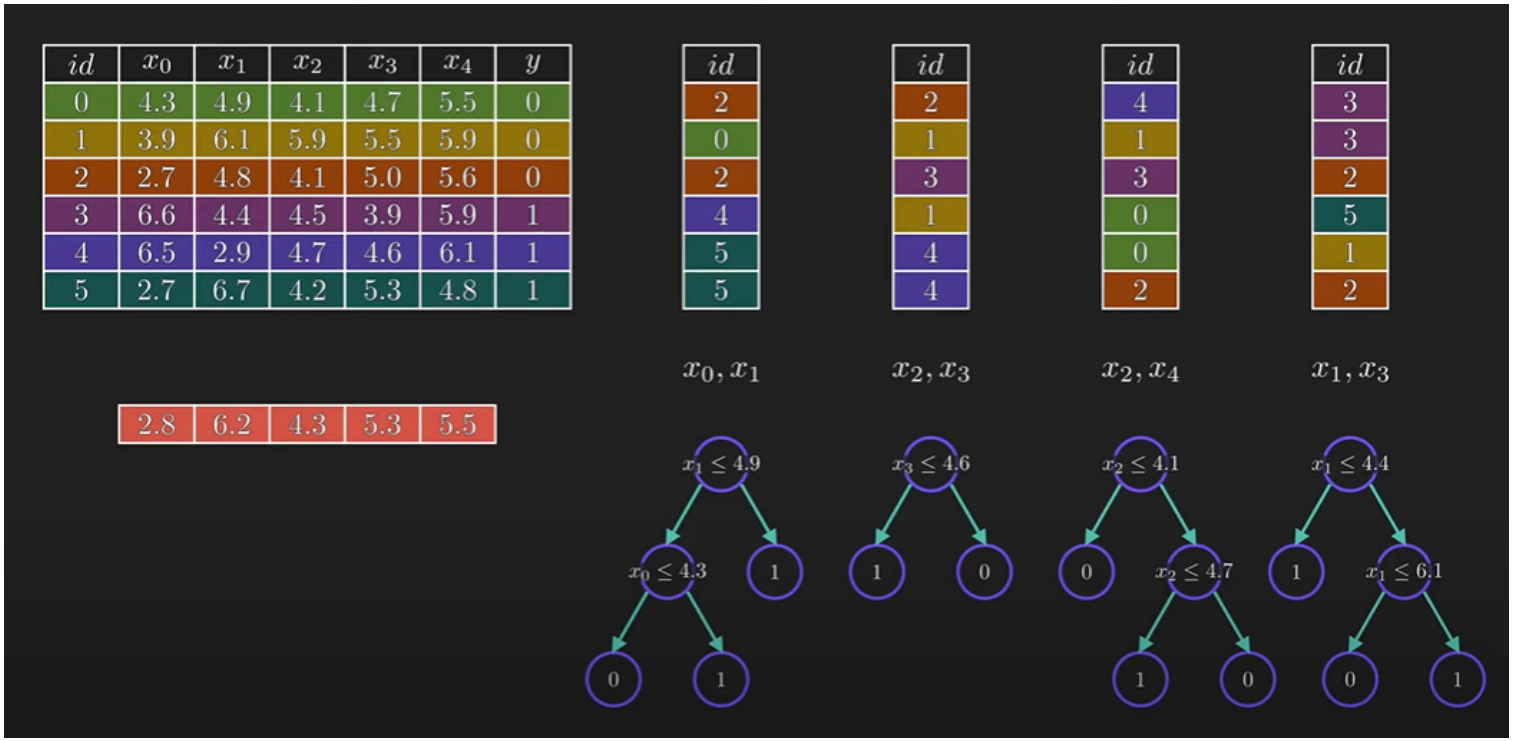
Stacking

- Multiple models are combined using another combiner model
  - Train models with training data and generate a new data set
  - New data set as input for the combiner model
  - Output of combiner model as final output
  - E.g., learns a better aggregation
- A parallel method and mainly reduces bias (complex models)



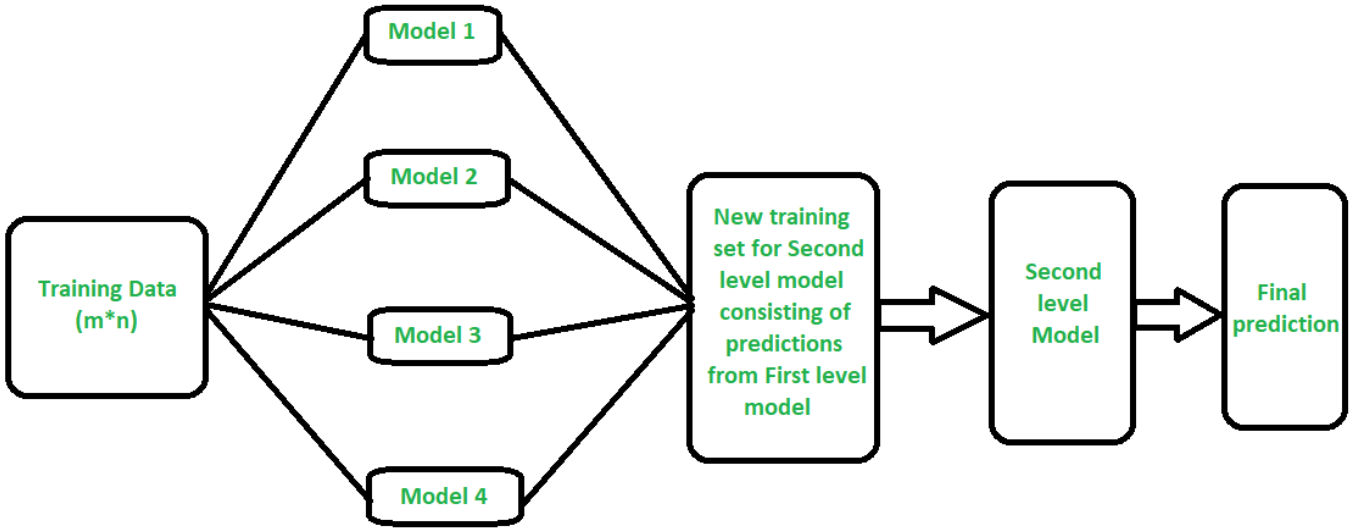
Random Forest

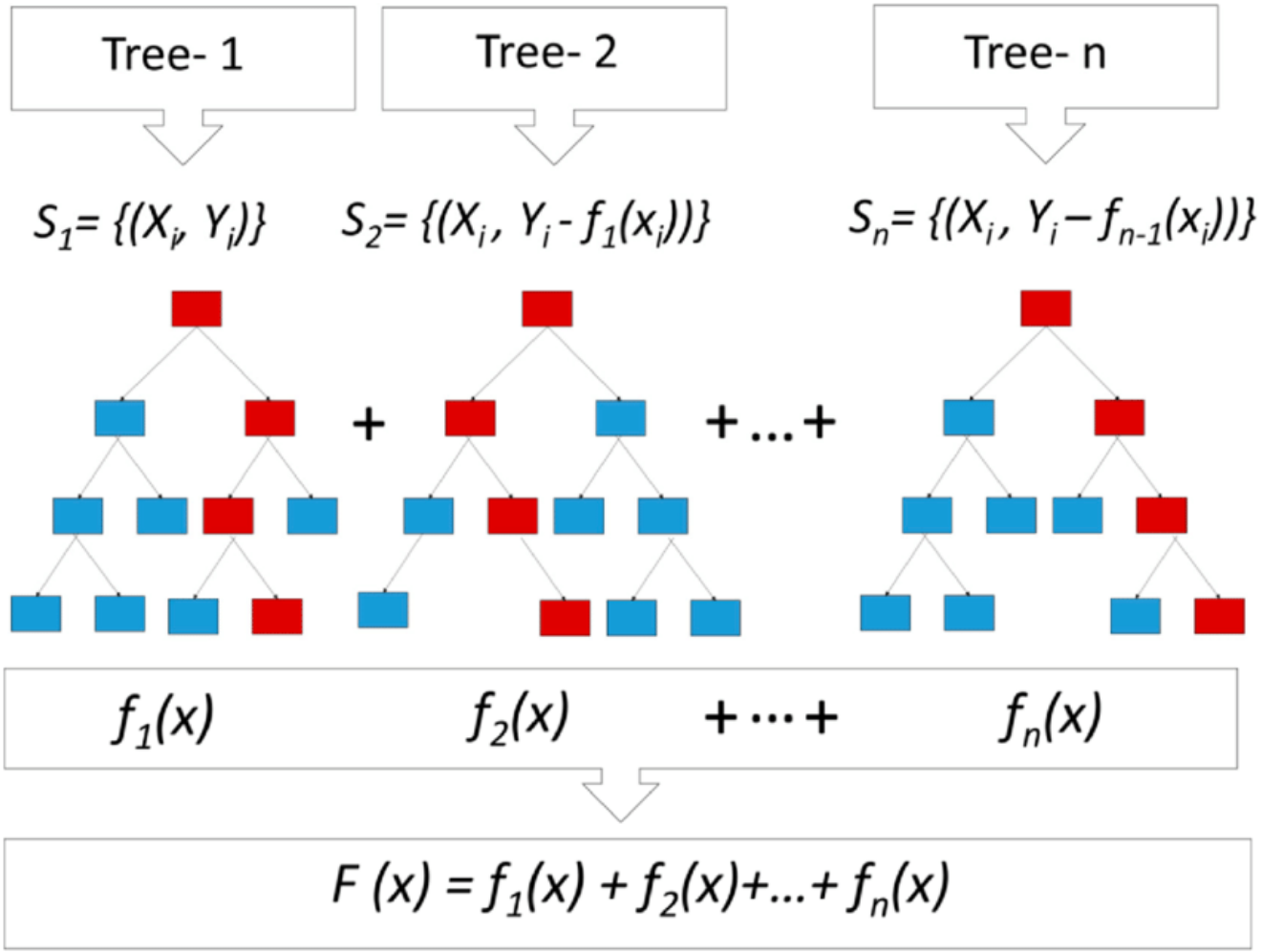
- Base learners are decision trees → ensemble becomes **forest**
- **Randomness**
  - Bootstrapped samples
  - Random subset of features → less correlation
- A parallel method and mainly reduces variance



Appendix: Stacking

- Multiple models are combined using another combiner model
  - Train models with training data and generate a new data set
  - New data set as input for the combiner model
  - Output of combiner model as final output
  - E.g., learns a better aggregation
- A parallel method and mainly reduces bias (complex models)



Gradient Boost

- Successive model predicts the residuals of preceding model
- Successive model aims to correct errors of preceding model
- A sequential method and mainly reduces bias